
REPORT ON PATH FINDING USING THE OPENLANEV2 API

August 20, 2025

MATHYS VINATIER - [GITHUB ASSIGNMENT PAGE](#)

1.1 Work with OpenLaneV2

1.1.1 Getting used to OpenLaneV2 features

The OpenLaneV2 offers multiple tools to obtain lane information. To familiarize ourselves with it, we created a script to plot the images captured by the car and the 2D map generated by the API for a given frame of a sequence.

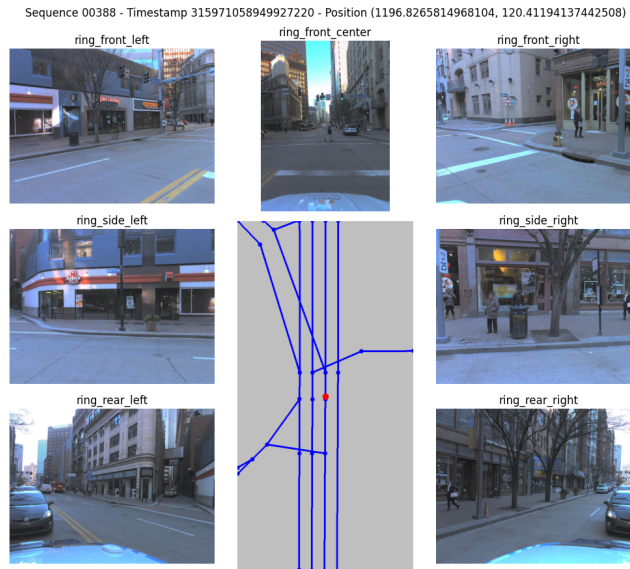


Figure 1.1: Example of the view for the first frame of sequence 00388

1.1.2 Creating a relationship graph

After getting used to the OpenLaneV2 library, I decided to use the **annotations** of the lane and the *topology_lsIs* to obtain the map of links between each centerline and extract the node relationships. After using the matrix, we were able to create a constellation map matching the node relationships and their IDs. To understand the relationships between nodes, we also built a dictionary to get the relative position of each node and plot the constellation using the *networkx* library, which is commonly used to create networks. On the right is an example of the plot for sequence 00388. After creating this graph, we started considering the path finding algorithm. However, the initial scripts that generated the constellation had issues, such as duplicated points. It was unclear whether this was a script error or if the OpenLaneV2 API was duplicating nodes with different IDs for some purpose. To address this, I implemented a *clean_nodes* function that merges duplicated points and their relationships into a unique node with a single ID. After building this map, we used all the frames of the sequence to continue exploration and update the constellation at each frame in case new nodes were found. One important point is that the node positions are given relative to the car. Later, we will need to compute the distance between points based on their geographic coordinates. To find the global position of each node, we added the global position of the car to perform a referential change whenever a node was found. After the last frame of the

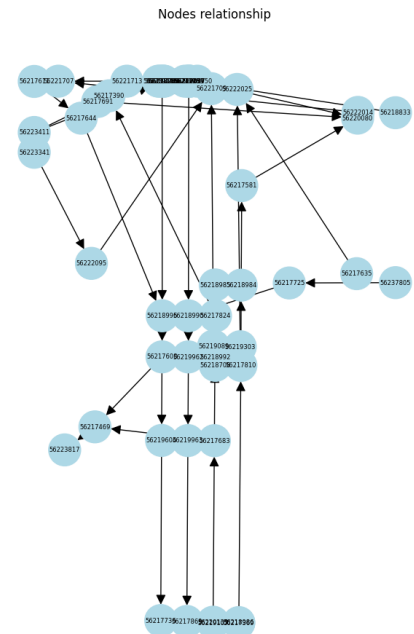


Figure 1.2: Constellation map of node relationships for sequence 00388

sequence, we clean the constellation again to avoid recurrent nodes. The exploration can be launched by the class method *start_exploration* of the Exploration object.

1.2 The Path Finding Algorithm

1.2.1 Choosing the algorithm

Two path finding algorithms have been suggested : Dijkstra and A*. I started exploring the options with the Dijkstra algorithm since I had previously used it in network classes during my cybersecurity studies. However, after reading about the A* algorithm, I found that it was better in terms of efficiency for path finding, especially, what I liked is that we can use a heuristic to get the actual cost of the path.

The Dijkstra algorithm computes the shortest path from a source node to all other nodes in a graph with non-negative edge weights. It does not use any heuristic and explores all possible paths, which can be expensive in terms of computation. Since I did not know how big could be the constellation I thought that this aspect of the algorithm could slow down the process. Also, I thought it would be more complicated to give path weight since it can depend on the distance in between actual and next node and to the main goal, since we are using direct coordinates I thought it would be harder to implement.

On the other hand, the A* algorithm uses a heuristic to estimate the cost from the current node directly to the goal that allow us to give a direction instead of blindly explore each nodes. This allowed me to use the Euclidean distance between the current node's geographic coordinates and the goal node's coordinates as the heuristic. Below is a step-by-step explanation of how the A* algorithm was implemented for path finding :

1. **Initialization :** The algorithm first identifies the closest nodes to the start and goal positions. These nodes serve as the entry and exit points for the path search.
2. **Graph Construction :** The weighted graph is built, where each node is connected to its neighbors with associated costs.
3. **Open List (Priority Queue) :** A priority queue is initialized to store nodes to be explored, prioritized by their estimated total cost $f = g + h$, where g is the cost from the start node and h is the heuristic estimate to the goal.
4. **Main Loop :** The algorithm avoid node with the lowest f -score from the queue. If the goal node is reached, the path is reconstructed by backpropagate through the nodes :
 - (a) **Neighbor Exploration :** For each neighbor of the current node, the algorithm calculates the tentative cost to reach it. If this path is better than any previously found, it updates the cost and adds the neighbor to the queue.
 - (b) **Path Reconstruction :** Once the goal is reached, the optimal path is reconstructed by following the recorded predecessors from the goal back to the start.
 - (c) **Failure Case :** If the queue is getting full without reaching the goal, the algorithm returns that no path was found.

1.2.2 Results of Path Finding

The `heapq` library is used to manage the priority queue. Using it simplifies the management of nodes and path cost values. The algorithm is implemented in the `A_star` method to compute the output path. After obtaining the path, we can use the previously defined method of the `Exploration` class to plot the start, the goal and the associated path. Here are the results for each sequence :

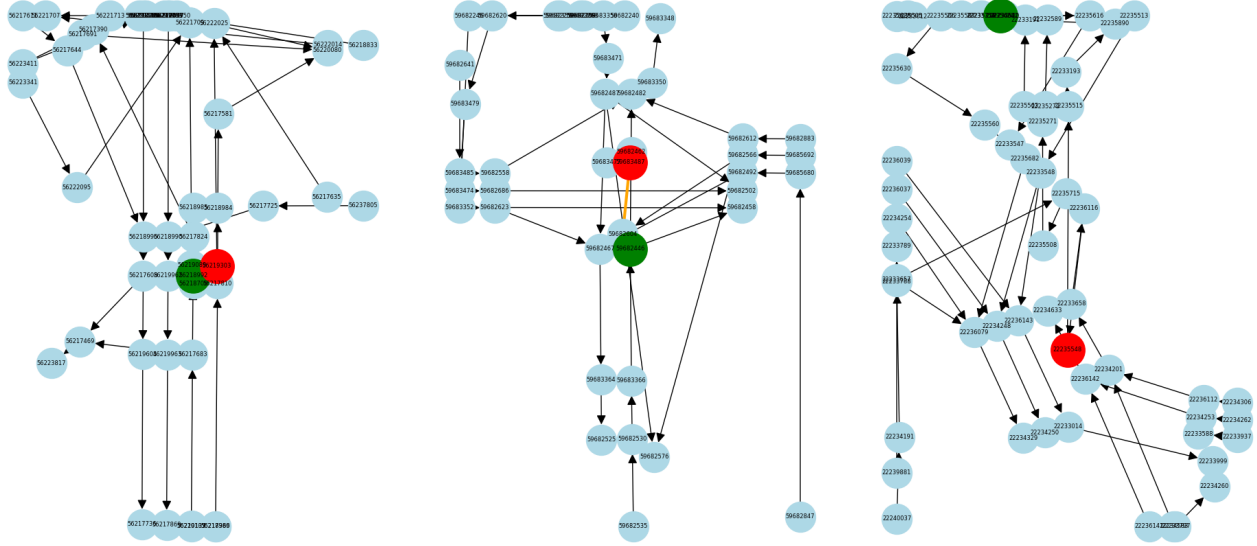


Figure 1.3: Paths for sequences 1, 2 and 3

We can see that only the path of sequence 2 has been found. After analyzing the results of sequences 1 and 3, I noticed that no path is possible in the given constellation. I concluded that the problem did not come from the pathfinding algorithm itself but from the constellation output.

1.3 Conclusion

In order to fix this issue, I would check the constellation scripts again using a global map of the actual road network to ensure that the computed paths are realistic. Also, for example, in sequence 1 we can see that the goal is located on the right lane relatively to the start. However, the issue in the constellation is that it does not consider interactions between different lane types. By incorporating the map of the *topology_lste*, we could allow for more possible connections and movements between parallel lanes.